

CSA0613 - Design and Analysis of Algorithms

CO4-AT1- Code Challenge

Divyesh S P (192424147)

Implementation Report: Longest Increasing Subsequence (LIS)

1. Aim

To implement the **Longest Increasing Subsequence (LIS)** problem using the **Dynamic Programming** technique and determine the maximum length of a strictly increasing subsequence.

2. Problem Statement

Given a sequence of integers, find the length of the longest subsequence in which the elements are arranged in strictly increasing order. The selected elements should maintain their original order, but they do not need to be adjacent.

Sample Input:

n = 6
arr = 10 22 9 33 21 50

Expected Output:

LIS Length = 4

One possible LIS is:

10 22 33 50

3. Algorithm

1. Read the number of elements and the sequence.
2. Create a Dynamic Programming array dp where every element is initially assigned 1.
3. For each element arr[i], compare it with all previous elements arr[j].
4. If arr[i] > arr[j], update:

```
dp[i] = max(dp[i], dp[j] + 1)
```

5. Find the maximum value in the dp array.
6. Display the LIS length.

4. Python Implementation

```
# Longest Increasing Subsequence using Dynamic Programming

# Input number of elements
n = int(input("Enter the number of elements: "))

# Input array elements
arr = list(map(int, input("Enter the sequence elements: ").split()))

# DP array: every element itself is an increasing subsequence of length 1
dp = [1] * n

# Calculate LIS length
for i in range(1, n):
    for j in range(i):
        if arr[i] > arr[j]:
            dp[i] = max(dp[i], dp[j] + 1)

# Find maximum value in dp
lis_length = max(dp)

# Output
print("LIS Length =", lis_length)
```

5. Output

```
Enter the number of elements: 6
Enter the sequence elements: 10 22 9 33 21 50
LIS Length = 4
```

6. Test Cases

Test Case	Input Sequence	Expected LIS Length	Result
Normal Case	10 22 9 33 21 50	4	Passed
Increasing Sequence	1 2 3 4 5	5	Passed
Decreasing Sequence	5 4 3 2 1	1	Passed

Test Case	Input Sequence	Expected LIS Length	Result
Single Element	10	1	Passed
Duplicate Values	2 2 2 2	1	Passed
Mixed Values	3 10 2 1 20	3	Passed

7. Complexity Analysis

- **Time Complexity:** $O(n^2)$
- **Space Complexity:** $O(n)$

The algorithm uses nested loops to compare each element with its previous elements. The Dynamic Programming array stores the LIS length ending at each position.

8. Conclusion

The Longest Increasing Subsequence problem was successfully implemented using **Dynamic Programming**. The program correctly identifies the maximum increasing subsequence length and handles different cases such as increasing, decreasing, duplicate, single-element, and mixed sequences. The implementation is efficient for moderate input sizes and follows the required algorithmic approach.